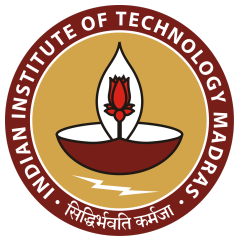

Machine Learning Project Bootcamp T1 2026

Day 3 : Data Preprocessing — Teaching Session

Presentation by : Piush Das

Contents for the Session

Concepts Covered

- Introduction to Data Preprocessing → Sets context and workflow
- Text Cleaning (Regex) → Raw text must be cleaned before any encoding or feature engineering
- Missing Values → Models cannot handle NaNs, so this is an early mandatory step
- Categorical Encoding → Convert non-numeric data to numeric form
- Feature Scaling & Normalization → Applied after data becomes numeric
- Time-based Preprocessing → Feature engineering from datetime columns
- Feature Selection → Remove irrelevant/redundant features after feature creation
- Dimensionality Reduction → Reduce feature space (e.g., PCA) after selection logic
- Pipeline (Chaining Transformers) → Final step to combine all preprocessing into a reproducible workflow

Demonstration [Day – 4]

- Applications of all concepts on e-commerce-review dataset

Introduction to Data Preprocessing

Preprocessing means transforming raw data into a form suitable for machine-learning models.

Why do we need preprocessing ?

- The real world training data is usually not clean and has many issues such as missing values for certain features, features on different scales, non-numeric attributes etc.
- Often there is a need to pre-process the data to make it amenable for training the model.
- The same pre-processing should be applied to both training and test set.

What are the key steps include in preprocessing ?

- Handling missing values
- Encoding categorical variables
- Feature scaling / normalization
- Parsing and manipulating time-based features
- Building a reproducible pipeline

What is the typical workflow ?

Acquire data → Load data → Exploratory Data Analysis → Text Cleaning → Handle Missing Values → Encode Categorical Features → Feature Scaling → Feature Extraction → Feature Selection → Split the Dataset → Model Building → Hyperparameter Training → Model Evaluation → Deployment

Basics of Regex and Text Cleaning

Text data in real-world datasets is often noisy and unstructured, containing HTML tags, URLs, emojis, punctuation, numbers, and inconsistent spacing. Before applying machine learning or NLP models, textual features must be cleaned and standardized so that models can extract meaningful patterns. Proper text preprocessing improves feature quality, reduces vocabulary size, and enhances model performance and generalization.

Why do we need text cleaning?

- Raw text contains noise such as HTML tags, links, special symbols, and extra spaces
- Machine learning models require consistent and structured input
- Cleaning improves tokenization, vectorization (TF-IDF, CountVectorizer), and embeddings
- Reduces sparsity and dimensionality in text features

Common Text Cleaning Steps

- Convert text to lowercase (standardization)
- Remove HTML tags (e.g., `<br>`, `<p>`)
- Remove URLs and email patterns
- Remove punctuation and special characters
- Remove numbers (if not contextually important)
- Remove extra whitespace and line breaks
- Optional: stopword removal, stemming, or lemmatization

Regular Expressions (Regex)

Regex is a powerful pattern-matching technique used for searching, extracting, and cleaning textual data efficiently. It allows us to define patterns for unwanted text such as HTML, digits, URLs, and multiple spaces.

Important Regex Patterns

- `<.*?>` → Removes HTML tags
- `http\S+|www\S+` → Removes URLs

- `\d+` → Removes digits
- `[^a-zA-Z\s]` → Removes special characters and punctuation
- `\s+` → Removes extra spaces and newlines
- `\n|\r|\t` → Removes line breaks and tabs

```
import re
import pandas as pd

def clean_text(text):
    # Convert to lowercase
    text = text.lower()

    # Remove HTML tags
    text = re.sub(r'<.*?>', '', text)

    # Remove URLs
    text = re.sub(r'http\S+|www\S+|https\S+', '', text)

    # Remove email addresses
    text = re.sub(r'\S+@\S+', '', text)

  # Remove numbers
    text = re.sub(r'\d+', '', text)

    # Remove special characters and punctuation
    text = re.sub(r'[^a-zA-Z\s]', '', text)
```

```
# Remove extra spaces, tabs, and newlines
    text = re.sub(r'\s+', ' ', text).strip()

    return text

# Example usage
df = pd.DataFrame({
    "text": [
        "<p>This is GREAT!!!</p> Visit https://example.com",
        "Contact me at test@gmail.com   \n New line here",
        "ML   is   AWESOME!!!   100% recommended <br>"
    ]
})

df["clean_text"] = df["text"].apply(clean_text)
print(df)
```

Note: The same text cleaning pipeline must be applied consistently to both training and test datasets to prevent data leakage and maintain reproducible preprocessing in machine learning workflows.

Handling Missing Values — Imputation

- Missing values occur due to errors in data capture such as sensor malfunctioning, measurement errors etc.
- Discarding records containing missing values would result in loss of valuable training samples.
- Many ML algorithms do not work with missing data and need all features to be present.

How to check for missing values ?

```
dataframe.isna().sum()
(dataframe.isna().sum() / len(dataframe))* 100
```

Imputation — Filling missing values

- For numerical variables: mean, median imputation.
- For categorical: mode, constant imputation.

Simple Imputer

```
import numpy as np
from sklearn.impute import SimpleImputer
imp_mean = SimpleImputer(missing_values=np.nan, strategy='mean')
imp_mean.fit([[7, 2, 3], [4, np.nan, 6], [10, 5, 9]])
X = [[np.nan, 2, 3], [4, np.nan, 6], [10, np.nan, 9]]
print(imp_mean.transform(X))
```

$$\text{imp_mean} = \begin{bmatrix} 7 & 2 & 3 \\ 4 & nan & 6 \\ 10 & 5 & 9 \end{bmatrix} \rightarrow \begin{bmatrix} \frac{7+4+10}{3} & \frac{2+5}{2} & \frac{3+6+9}{3} \end{bmatrix} \rightarrow \begin{bmatrix} 7.0 & 3.5 & 6.0 \end{bmatrix}$$

$$\text{X} = \begin{bmatrix} nan & 2 & 3 \\ 4 & nan & 6 \\ 10 & nan & 9 \end{bmatrix}$$

$$\text{imp_mean.transform(X)} = \begin{bmatrix} 7.0 & 2.0 & 3.0 \\ 4.0 & 3.5 & 6.0 \\ 10.0 & 3.5 & 9.0 \end{bmatrix}$$

Other strategy :

- If “mean”, then replace missing values using the mean along each column. Can only be used with numeric data.
- If “median”, then replace missing values using the median along each column. Can only be used with numeric data.
- If “most_frequent”, then replace missing using the most frequent value along each column. Can be used with strings or numeric data.
- If “constant”, then replace missing values with fill_value. Can be used with strings or numeric data.

When to Drop vs When to Impute ?

- Drop a feature when it has a large percentage of missing values (typically over 50%) or adds little value to the prediction task.
- Impute missing values when the feature is important or moderately incomplete, using mean, median, or mode as appropriate.

Additional : You can also have a look at *Iterative Imputer* and *KNN Imputer*

Handling Categorical Features — Encoding

All Machine Learning algorithms require numeric input. Categorical variables must be encoded into numeric form.

Onehot Encoder

- Encodes categorical feature or label as a one-hot numeric array.
- Creates one binary column for each of unique values.
- Exactly one column has 1 in it and rest have 0.

$$x' = [0, \ \ldots \ 1 \ \ldots \ , 0] \text{ for category } C_i$$

```
from sklearn.preprocessing import OneHotEncoder
data = [['red'], ['green'], ['blue']]
encoder = OneHotEncoder()
print(encoder.fit(data))
```

⋙ OneHotEncoder()

```
print(encoder.transform(data).toarray())
```

⋙
$$\begin{bmatrix} 0.0 & 0.0 & 1.0 \\ 0.0 & 1.0 & 0.0 \\ 1.0 & 0.0 & 0.0 \end{bmatrix}$$

```
print(encoder.categories_)
```

⋙ [array(['blue', 'green', 'red'], dtype=object)]

Label Encoder

- Converts categorical string labels into numeric form (0, 1, 2, …).
- Useful for target variables or ordinal features where label order doesn't matter.
- Each unique category is mapped to an integer.

$$x' = \text{LabelEncoder}(x)$$

```
from sklearn.preprocessing import LabelEncoder
data = ['red', 'green', 'blue', 'green', 'red']
encoder = LabelEncoder()
print(encoder.fit(data))
```

⋙ LabelEncoder()

- here k = 3 {'blue' , 'green' , 'red'}
- blue is encoded as 0 , green as 1 , read as 2

```
print(encoder.transform(data))
```

⋙ [2 1 0 1 2]

```
print(encoder.inverse_transform([0, 1, 2]))
```

⋙ ['blue' 'green' 'red']

Ordinal Encoder

- Converts categorical data into integer values representing order/rank.
- Must specify or assume a meaningful order among categories.
- Often used for features like “low”, “medium”, “high”.

$$x' = \text{rank(category)}$$

```
from sklearn.preprocessing import OrdinalEncoder
data = [['low'], ['medium'], ['high'], ['medium']]
encoder = OrdinalEncoder(categories=[['low', 'medium', 'high']])
print(encoder.fit(data))
```

⋙ OrdinalEncoder(categories=[['low', 'medium', 'high']])

```
print(encoder.transform(data))
```

⋙
$$\begin{bmatrix} 0.0 \\ 1.0 \\ 2.0 \\ 1.0 \end{bmatrix}$$

```
print(encoder.inverse_transform([[0], [2]]))
```

⋙ [['low']
['high']]

Additional : You can also have a look at *Target Encoding* and *Frequency Encoding*

Comparison of Encoding Techniques

The LabelEncoder, OneHotEncoder, and OrdinalEncoder are all techniques used to convert categorical data into numerical form before feeding it into machine learning models. Although they serve the same purpose - transforming categories into numbers - they work differently and are suited for different scenarios depending on the type of categorical variable.

- LabelEncoder assigns each unique category a numerical label such as red → 2, green → 1, and blue → 0. It is simple and compact, making it useful for encoding target variables (y) or when using tree-based algorithms like Decision Trees and Random Forests, which are not sensitive to numeric ordering. However, in models that interpret numerical relationships like Linear Regression or SVM, it can be misleading, as the encoded numbers may imply an artificial hierarchy or distance between categories.

- OneHotEncoder creates new binary columns for every unique category, assigning a value of 1 if the sample belongs to that category and 0 otherwise. This encoding is ideal for nominal categorical variables - features without any natural order, such as gender, city, or color. OneHotEncoding prevents the model from inferring an incorrect ordinal relationship among categories, ensuring equal importance for all. However, it can lead to a “curse of dimensionality” when applied to features with many unique categories, increasing both memory usage and computation time.

- OrdinalEncoder converts categories into integers based on their inherent order, such as low → 0, medium → 1, high → 2. It preserves ordinal relationships, making it suitable for ordered categorical data, where the rank of categories carries meaning (e.g., education level or satisfaction rating). However, this encoding assumes equal spacing between categories, which may not always represent the real-world relationship accurately.

In summary, use LabelEncoder for encoding simple categorical targets or features in tree-based models, OneHotEncoder for unordered categorical data where no ranking exists, and OrdinalEncoder for ordered features where the sequence is meaningful. Choosing the correct encoding method ensures that your model interprets categorical information appropriately without introducing bias or distortion.

Handling Numerical Features – Feature Scaling & Normalization

Standard Scaler

- Transforms the original features vector into new feature vector using following formula

$$z = \frac{x - \mu}{\sigma}$$

- Standardize features by removing the mean and scaling to unit variance.

```
from sklearn.preprocessing import StandardScaler
data = [[0, 0], [0, 0], [1, 1], [1, 1]]
scaler = StandardScaler()
print(scaler.fit(data))
```

⋙ StandardScaler()

$$z = \frac{x - 0.5}{0.5}$$

```
print(scaler.transform(data))
```

⋙
$$\begin{bmatrix} -1.0 & -1.0 \\ -1.0 & -1.0 \\ 1.0 & 1.0 \\ 1.0 & 1.0 \end{bmatrix}$$

```
print(scaler.transform([[2, 2]]))
```

⋙ [[3.0 3.0]]

MinMax Scaler

- Transform features by scaling each feature to a given range.
- This estimator scales and translates each feature individually such that it is in the given range on the training set, e.g. between zero and one.

$$x' = \frac{x - x.min}{x.max - x.min}$$

```
from sklearn.preprocessing import MinMaxScaler
data = [[-1, 2], [-0.5, 6], [0, 10], [1, 18]]
scaler = MinMaxScaler()
print(scaler.fit(data))
```

⋙ MinMaxScaler()

$$x' = \frac{x - [-1.0\ \ 2.0]}{[1.0\ \ 18.0] - [-1.0\ \ 2.0]} = \frac{x\ -\ [-1.0\ \ 2.0]}{[2.0\ \ 16.0]}$$

```
print(scaler.transform(data))
```

⋙
$$\begin{bmatrix} 0.0 & 0.0 \\ 0.25 & 0.25 \\ 0.5 & 0.5 \\ 1.0 & 1.0 \end{bmatrix}$$

```
print(scaler.transform([[2, 2]]))
```

⋙[[1.5 0.0]]

MaxAbs Scaler

- Scale each feature by its maximum absolute value.
- This estimator scales and translates each feature individually such that the maximal absolute value of each feature in the training set will be 1.0.
- MaxAbsScaler doesn't reduce the effect of outliers.

$$x' = \frac{x}{\max(x.\max , |x.\min|)}$$

```
from sklearn.preprocessing import MaxAbsScaler
X = [[ 1., -1.,  2.],
      [ 2.,  0.,  0.],
      [ 0.,  1., -1.]]
transformer = MaxAbsScaler()
print(transformer.fit(X))
```

⋙ MaxAbsScaler()

$$x' = \frac{x}{[2.0 \;\; 1.0 \;\; 2.0]}$$

```
transformer.transform(X)
```

⋙
$$\begin{bmatrix} 0.5 & -1.0 & 1.0 \\ 1.0 & 0.0 & 0.0 \\ 0.0 & 1.0 & -0.5 \end{bmatrix}$$

```
ransformer.transform([[2 , 2 , 2]])
```

⋙ [[1.0 2.0 1.0]]

Comparison of Data Scaling Techniques

The StandardScaler, MinMaxScaler, and MaxAbsScaler are all normalization techniques used to scale features before feeding them into machine learning models, but they work differently and are suited for different scenarios. We scale data to ensure that all numerical features contribute equally during model training. Since many ML algorithms are sensitive to feature magnitude, unscaled data can cause large-valued features to dominate learning. Scaling improves model performance, stability, and convergence speed.

- The StandardScaler standardizes data by removing the mean and scaling it to unit variance, meaning each feature will have a mean of 0 and a standard deviation of 1. It is ideal when features follow a roughly normal (Gaussian) distribution and when algorithms assume standardized input, such as Linear Regression, Logistic Regression, SVM, PCA, and K-Means. However, it is sensitive to outliers because the mean and standard deviation are affected by extreme values.

- The MinMaxScaler rescales data to a fixed range, typically between 0 and 1, using the minimum and maximum values of each feature. This method preserves the shape of the data distribution but compresses all features into the same range. It is particularly useful for algorithms that rely on distance or gradient calculations - such as Neural Networks, KNN, and K-Means - because it ensures all features contribute equally. However, it is also sensitive to outliers, as one extreme value can distort the scaling.

- The MaxAbsScaler scales each feature by dividing by its maximum absolute value, bringing the range to [-1, 1] without shifting or centering the data. This makes it well-suited for datasets that contain both positive and negative values and especially for sparse data, since it preserves zero entries. Unlike StandardScaler, it doesn't change the data's mean or sparsity.

In summary, use StandardScaler when your features are normally distributed, MinMaxScaler when you need all features in the same range for gradient or distance-based models, and MaxAbsScaler when dealing with sparse or signed data. If the dataset has many outliers, it's better to use ***RobustScaler***, which scales using the median and interquartile range instead of the mean and standard deviation.

Time-based preprocessing

Parsing dates

If our dataset has date/time columns, we need to convert them into datetime objects and extract useful features: year, month, day, day of week, hour etc.

```
df = pd.read_csv("train.csv")

# Parse the date column
df['date'] = pd.to_datetime(df['date'], format='%Y-%m-%d')

# Set the date as the index for time-series operations
df.set_index('date', inplace=True)
```

- pd.to_datetime() converts string dates to pandas datetime objects.
- Setting the index to the date column allows easy time-based slicing and resampling.

```
df['year'] = df.index.year
df['month'] = df.index.month
df['day'] = df.index.day
df['day_of_week'] = df.index.dayofweek
df['is_weekend'] = df['day_of_week'].isin([5, 6]).astype(int)

df.head()
```

- These new features help machine learning models recognize temporal patterns like: trend and seasonality

Resampling

- Resampling means aggregating data over a new time frequency (e.g., weekly, monthly, quarterly).
- Useful for smoothing noise or analyzing long-term trends.

```
df["weekly_avg"] = df['meantemp'].resample('W').mean()
df["monthly_humidity"] = df['humidity'].resample('ME').mean()
df.head(31)
```

- 'W' → weekly frequency
- 'ME' → month-end frequency
- 'Q' → quarterly
- 'Y' → yearly

Handling Missing Date

- Sometimes time-series data may have missing dates. we can detect and fill them for consistency.

```
# Create a continuous date range
full_range = pd.date_range(start=df.index.min(), end=df.index.max())

# Reindex and forward-fill missing entries
df = df.reindex(full_range)
df.ffill(inplace=True)

print("After filling missing dates:", df.shape)
```

- pd.date_range() creates a continuous date sequence.
- Reindexing ensures every date is present.
- ffill() fills missing values with the last known observation.

Lag Features

- Lag features represent past values of a variable at previous time steps
- Useful for forecasting and time-series modeling
- Example: Sales at time (t-1), (t-2), etc.

```
import pandas as pd

# Sample time-series dataset
data = {
   "date": pd.date_range(start="2024-01-01", periods=10, freq="D"),
   "sales": [100, 120, 130, 90, 150, 170, 160, 180, 200, 210]
}

df = pd.DataFrame(data)
df.set_index("date", inplace=True)

# Creating Lag Features
df["sales_lag_1"] = df["sales"].shift(1)
df["sales_lag_2"] = df["sales"].shift(2)
```

Rolling Statistics (Window Features)

- Rolling Mean: Smooths short-term fluctuations and highlights trends
- Rolling Standard Deviation: Measures variability over time
- Rolling Sum/Min/Max: Captures local temporal patterns
- Window size determines how many past observations are used

```
# Creating Rolling Features
df["rolling_mean_3"] = df["sales"].rolling(window=3).mean()
df["rolling_std_3"] = df["sales"].rolling(window=3).std()

print(df)
```

Example Interpretation

- sales_lag_1 → Sales from previous day
- sales_lag_2 → Sales from two days before
- rolling_mean_3 → Average sales over last 3 days (trend smoothing)
- rolling_std_3 → Variation in sales over last 3 days

Note: Lag and rolling features introduce missing values at the beginning of the series, which should be handled using appropriate imputation or row dropping. These transformations must be applied after sorting data by time and consistently used on both training and test datasets to maintain temporal integrity and avoid data leakage.

Feature Selection

Feature Selection and Dimensionality Reduction are essential preprocessing techniques used to reduce the number of input features while preserving the most important information for model learning. Real-world datasets often contain redundant, irrelevant, or highly correlated features that can increase model complexity, training time, and risk of overfitting. Selecting the most relevant features improves interpretability, computational efficiency, and model generalization.

Why do we need Feature Selection?

- Reduces overfitting by removing noisy and irrelevant features
- Improves model performance and training speed
- Enhances interpretability of the model
- Handles the curse of dimensionality in high-dimensional datasets

Types of Feature Selection Methods

- Filter Methods: Select features based on statistical measures (correlation, variance, chi-square)
- Wrapper Methods: Use model performance to evaluate feature subsets (RFE)
- Embedded Methods: Feature selection occurs during model training (Lasso, Tree-based models)

```
from sklearn.feature_selection import VarianceThreshold
import pandas as pd

# Example dataset
X = pd.DataFrame({
    "A": [1, 1, 1, 1, 1],
    "B": [1, 2, 3, 4, 5],
    "C": [2, 2, 2, 2, 2]
})

# Remove low variance features
selector = VarianceThreshold(threshold=0.0)
X_selected = selector.fit_transform(X)

print("Selected Features Shape:", X_selected.shape)
```

Dimensionality Reduction

Dimensionality Reduction transforms the original feature space into a lower-dimensional space while retaining maximum variance or information. Unlike feature selection, it creates new features instead of selecting existing ones.

Principal Component Analysis (PCA)

- PCA is a linear dimensionality reduction technique
- Projects data onto orthogonal components with maximum variance
- Helps in noise reduction and visualization of high-dimensional data
- Sensitive to feature scaling, hence scaling should be applied before PCA

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import pandas as pd

# Sample numerical dataset
X = pd.DataFrame({
    "feature1": [2, 4, 6, 8, 10],
    "feature2": [1, 3, 5, 7, 9],
    "feature3": [10, 20, 30, 40, 50]
})

# Step 1: Scale the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
# Step 2: Apply PCA
pca = PCA(n_components=2)
X_reduced = pca.fit_transform(X_scaled)

print("Original Shape:", X.shape)
print("Reduced Shape:", X_reduced.shape)
```

Note: Feature scaling should be performed before applying PCA, and the same selected features or transformation must be consistently applied to both training and test datasets within a preprocessing pipeline to avoid data leakage.

Chaining Transformers – Building Pipeline

A Pipeline in scikit-learn is a powerful tool that allows us to chain multiple preprocessing steps and a model into a single workflow. It ensures that the same transformations applied during training are automatically applied to new data during prediction, preventing data leakage and making the process clean, consistent, and easy to manage. A ColumnTransformer is used within a pipeline to apply different transformations to different columns - for example, scaling numerical features while one-hot encoding categorical ones. Together, they create an organized and reproducible machine learning workflow where preprocessing and modeling happen seamlessly in one step.

Column Transformer

- It applies a set of transformers to columns of an array or pandas.DataFrame, concatenates the transformed outputs from different transformers into a single matrix.
- It is useful for transforming heterogenous data by applying different transformers to separate subsets of features.
- It combines different feature selection mechanisms and transformation into a single transformer object.

```
preprocessor = ColumnTransformer(
   transformers=[('num', StandardScaler(), num_features),
    ('cat', OneHotEncoder(), cat_features)
    ])
```

Pipeline

- Constructs a chain of multiple transformers to execute a fixed sequence of steps in data preprocessing and modelling.

```
pipeline = Pipeline(steps=[
   ('preprocessing', preprocessor),
   ('model', LogisticRegression())
])
```

Thank You !